Lab 1: Python Data Tools

Course: IT7103 Advanced AI Applications

Tools: Python, NumPy, Pandas, Matplotlib,and Seaborn

Dataset: Iris flower dataset

1. Setup

Run this cell first. If a package is missing in your local environment, install it before continuing.

Code cell

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris

sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")
```

Output

```
Libraries imported successfully.
```

2. Part A - Python revision

This part revises variables, lists, dictionaries, conditions, functions, and loops.

Code cell

```
student_name = "Sara"
student_id = 2026001
lab_mark = 8.5
passed = lab_mark >= 5

print("Student:", student_name)
print("ID:", student_id)
print("Mark:", lab_mark)
print("Passed:", passed)
```

Output

```
Student: Sara
ID: 2026001
Mark: 8.5
Passed: True
```

Task A1: Create three variables for yourself: `your_name`, `your_id`, and `expected_mark`. Print them in one sentence.

Question A1: What is the difference between a string, an integer, a float, and a Boolean value?

Code cell

```
# TODO: Complete Task A1 here

your_name = "Type your name here"
your_id = 0
expected_mark = 0.0

print(f"My name is {your_name}, my ID is {your_id}, and my expected mark is {expected_mark}.")
```

Output

```
My name is Type your name here, my ID is 0, and my expected mark is 0.0.
```

Code cell

```
features = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
species_count = {"setosa": 50, "versicolor": 50, "virginica": 50}

print(features[0])
print(features[-1])
print(species_count["setosa"])
```

Output

```
sepal_length
petal_width
50
```

Task A2: Add a new item called `species` to the `features` list.

Task A3: Add a new key called `total_samples` to the `species_count` dictionary with the value `150`.

Question A2: When would you choose a dictionary instead of a list?

Code cell

```
# TODO: Complete Tasks A2 and A3 here

features.append("species")
species_count["total_samples"] = 150

print(features)
print(species_count)
```

Output

```
['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
{'setosa': 50, 'versicolor': 50, 'virginica': 50, 'total_samples': 150}
```

Code cell

```
def describe_mark(mark):
    if mark >= 8:
        return "Excellent"
    elif mark >= 5:
        return "Pass"
    else:
        return "Needs improvement"

for mark in [4, 6, 9]:
    print(mark, describe_mark(mark))
```

Output

```
4 Needs improvement
6 Pass
9 Excellent
```

Task A4: Modify the function so that marks from 7 to less than 8 return `Good`.

Task A5: Write a function called `is_valid_measurement(value)` that returns `True` if the value is greater than zero.

Code cell

```
# TODO: Complete Tasks A4 and A5 here

def describe_mark_v2(mark):
    if mark >= 8:
        return "Excellent"
    elif mark >= 7:
        return "Good"
    elif mark >= 5:
        return "Pass"
    else:
        return "Needs improvement"

def is_valid_measurement(value):
    return value > 0

for mark in [4, 6, 7.5, 9]:
    print(mark, describe_mark_v2(mark))

print(is_valid_measurement(3.5))
print(is_valid_measurement(-1))
```

Output

```
4 Needs improvement
6 Pass
7.5 Good
9 Excellent
True
False
```

Code cell

```
marks = [6, 7.5, 9, 4, 8]

passed_count = 0
for mark in marks:
    if mark >= 5:
        passed_count += 1

print("Number of passing marks:", passed_count)
print("Average mark:", sum(marks) / len(marks))
```

Output

```
Number of passing marks: 4
Average mark: 6.9
```

3. Part B - NumPy revision

NumPy helps us work with numerical arrays. In later AI lectures, features and model inputs are often represented as arrays.

Code cell

```
measurements = np.array([
    [5.1, 3.5, 1.4, 0.2],
    [4.9, 3.0, 1.4, 0.2],
    [6.2, 3.4, 5.4, 2.3],
    [5.9, 3.0, 5.1, 1.8]
])

print("Array:")
print(measurements)
print("Shape:", measurements.shape)
print("Data type:", measurements.dtype)
```

Output

```
Array:
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [6.2 3.4 5.4 2.3]
 [5.9 3.  5.1 1.8]]
Shape: (4, 4)
Data type: float64
```

Task B1: How many rows and columns are in this array?

Task B2: Print the data type of the array using `measurements.dtype`.

Code cell

```
print("Means:", measurements.mean(axis=0))
print("Minimum values:", measurements.min(axis=0))
print("Maximum values:", measurements.max(axis=0))
print("Standard deviation:", measurements.std(axis=0))
```

Output

```
Means: [5.525 3.225 3.325 1.125]
Minimum values: [4.9 3.  1.4 0.2]
Maximum values: [6.2 3.5 5.4 2.3]
Standard deviation: [0.54025457 0.22776084 1.92791986 0.94174041]
```

Task B3: Calculate the standard deviation of each column using `.std(axis=0)`.

Task B4: Select only the petal length column.

Code cell

```
petal_length = measurements[:, 2]
petal_width = measurements[:, 3]
petal_area = petal_length * petal_width

sepal_length = measurements[:, 0]
sepal_width = measurements[:, 1]
sepal_area = sepal_length * sepal_width
petal_ratio = petal_length / petal_width

print("Petal length:", petal_length)
print("Approximate petal area:", petal_area)
print("Approximate sepal area:", sepal_area)
print("Petal ratio:", petal_ratio)
```

Output

```
Petal length: [1.4 1.4 5.4 5.1]
Approximate petal area: [ 0.28  0.28 12.42  9.18]
Approximate sepal area: [17.85 14.7  21.08 17.7 ]
Petal ratio: [7.         7.         2.34782609 2.83333333]
```

Question B1: Why is vectorized calculation useful when working with larger datasets?

4. Part C - Loading real data with Pandas

Now we load the Iris dataset into a Pandas DataFrame and practise inspection, filtering, grouping, and feature creation.

Code cell

```
iris = load_iris(as_frame=True)
df = iris.frame.copy()

df["species"] = df["target"].map(dict(enumerate(iris.target_names)))
df = df.drop(columns=["target"])
df.columns = ["sepal_length", "sepal_width", "petal_length", "petal_width", "species"]

df.head()
```

Output

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Code cell

```
df.head(10)
```

Output

```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
5           5.4          3.9           1.7          0.4  setosa
6           4.6          3.4           1.4          0.3  setosa
7           5.0          3.4           1.5          0.2  setosa
8           4.4          2.9           1.4          0.2  setosa
9           4.9          3.1           1.5          0.1  setosa
```

Code cell

```
df.tail()
```

Output

```
     sepal_length  sepal_width  petal_length  petal_width    species
145           6.7          3.0           5.2          2.3  virginica
146           6.3          2.5           5.0          1.9  virginica
147           6.5          3.0           5.2          2.0  virginica
148           6.2          3.4           5.4          2.3  virginica
149           5.9          3.0           5.1          1.8  virginica
```

Code cell

```
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
print("Missing values:")
print(df.isna().sum())
```

Output

```
Shape: (150, 5)
Columns: ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
Missing values:
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

Code cell

```
df.info()
```

Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

Code cell

```
df.describe()
```

Output

```
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.057333      3.758000     1.199333
std        0.828066     0.435866      1.765298     0.762238
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000
```

Task C3: How many rows and columns are in the final DataFrame?

Task C4: Are there any missing values?

Question C1: Why should we inspect a dataset before using it in an AI or data-analysis task?

Code cell

```
large_petals = df[df["petal_length"] > 5.0]
wide_sepals = df[df["sepal_width"] > 3.2]
setosa_only = df[df["species"] == "setosa"]

print("Large petals:", large_petals.shape)
print("Wide sepals:", wide_sepals.shape)
print("Setosa only:", setosa_only.shape)
```

Output

```
Large petals: (42, 5)
Wide sepals: (43, 5)
Setosa only: (50, 5)
```

Task C5: Create a DataFrame that contains only flowers where `petal_width` is greater than 1.5.

Task C6: Create a DataFrame that contains only `virginica` flowers.

Task C7: Create a DataFrame that contains flowers with `sepal_length` greater than 6 and `petal_width` greater than 1.5.

Code cell

```
# TODO: Complete Tasks C5, C6, and C7 here

wide_petals = df[df["petal_width"] > 1.5]
virginica_only = df[df["species"] == "virginica"]
combined_filter = df[(df["sepal_length"] > 6) & (df["petal_width"] > 1.5)]

print(wide_petals.shape)
print(virginica_only.shape)
print(combined_filter.shape)
```

Output

```
(52, 5)
(50, 5)
(41, 5)
```

Code cell

```
df["petal_area"] = df["petal_length"] * df["petal_width"]
df["sepal_area"] = df["sepal_length"] * df["sepal_width"]
df["petal_to_sepal_area"] = df["petal_area"] / df["sepal_area"]
df["length_difference"] = df["petal_length"] - df["sepal_length"]

df.head()
```

Output

```
   sepal_length  sepal_width  petal_length  petal_width species  petal_area  \
0           5.1          3.5           1.4          0.2  setosa        0.28
1           4.9          3.0           1.4          0.2  setosa        0.28
2           4.7          3.2           1.3          0.2  setosa        0.26
3           4.6          3.1           1.5          0.2  setosa        0.30
4           5.0          3.6           1.4          0.2  setosa        0.28

   sepal_area  petal_to_sepal_area  length_difference
0       17.85             0.015686               -3.7
1       14.70             0.019048               -3.5
2       15.04             0.017287               -3.4
3       14.26             0.021038               -3.1
4       18.00             0.015556               -3.6
```

Question C2: In your own words, what is a derived feature?

Code cell

```
df_sorted = df.sort_values(by="petal_area", ascending=False)
df_sorted.head(10)
```

Output

```
     sepal_length  sepal_width  petal_length  petal_width    species  \
118           7.7          2.6           6.9          2.3  virginica
109           7.2          3.6           6.1          2.5  virginica
100           6.3          3.3           6.0          2.5  virginica
117           7.7          3.8           6.7          2.2  virginica
144           6.7          3.3           5.7          2.5  virginica
135           7.7          3.0           6.1          2.3  virginica
105           7.6          3.0           6.6          2.1  virginica
143           6.8          3.2           5.9          2.3  virginica
140           6.7          3.1           5.6          2.4  virginica
136           6.3          3.4           5.6          2.4  virginica

     petal_area  sepal_area  petal_to_sepal_area  length_difference
118       15.87       20.02             0.792707               -0.8
109       15.25       25.92             0.588349               -1.1
100       15.00       20.79             0.721501               -0.3
117       14.74       29.26             0.503759               -1.0
144       14.25       22.11             0.644505               -1.0
135       14.03       23.10             0.607359               -1.6
105       13.86       22.80             0.607895               -1.0
143       13.57       21.76             0.623621               -0.9
140       13.44       20.77             0.647087               -1.1
136       13.44       21.42             0.627451               -0.7
```

Code cell

```
species_summary = df.groupby("species")[[
    "sepal_length", "sepal_width", "petal_length", "petal_width", "petal_area"
]].mean()

species_summary
```

Output

```
            sepal_length  sepal_width  petal_length  petal_width  petal_area
species
setosa             5.006        3.428         1.462        0.246      0.3656
versicolor         5.936        2.770         4.260        1.326      5.7204
virginica          6.588        2.974         5.552        2.026     11.2962
```

Code cell

```
max_sepal_length = df.groupby("species")["sepal_length"].max()
print(max_sepal_length)
```

Output

```
species
setosa        5.8
versicolor    7.0
virginica     7.9
Name: sepal_length, dtype: float64
```

Task C9: Which species has the largest average petal area?

Task C10: Use `groupby()` to calculate the maximum `sepal_length` for each species.

Question C3: Why is grouping useful when comparing categories in a dataset?

5. Part D - Visualization with Matplotlib

Matplotlib gives direct control over figures, labels, and chart elements.

Code cell

```
plt.figure(figsize=(6, 4))
plt.scatter(df["sepal_length"], df["petal_length"])
plt.xlabel("Sepal length (cm)")
plt.ylabel("Petal length (cm)")
plt.title("Sepal length vs petal length")
plt.show()
```

Output

```
<Figure size 600x400 with 1 Axes>
```

Code cell

```
# Task D1: scatter plot comparing petal_length and petal_width
plt.figure(figsize=(6, 4))
plt.scatter(df["petal_length"], df["petal_width"])
plt.xlabel("Petal length (cm)")
plt.ylabel("Petal width (cm)")
plt.title("Petal length vs petal width")
plt.show()
```

Output

```
<Figure size 600x400 with 1 Axes>
```

Code cell

```
plt.figure(figsize=(6, 4))
plt.hist(df["sepal_length"], bins=12)
plt.xlabel("Sepal length (cm)")
plt.ylabel("Frequency")
plt.title("Distribution of sepal length")
plt.show()
```

Output

```
<Figure size 600x400 with 1 Axes>
```

Code cell

```
# Task D2: histogram for petal_width
plt.figure(figsize=(6, 4))
plt.hist(df["petal_width"], bins=12)
plt.xlabel("Petal width (cm)")
plt.ylabel("Frequency")
plt.title("Distribution of petal width")
plt.show()
```

Output

```
<Figure size 600x400 with 1 Axes>
```

Code cell

```
species_counts = df["species"].value_counts()

plt.figure(figsize=(6, 4))
plt.bar(species_counts.index, species_counts.values)
plt.xlabel("Species")
plt.ylabel("Number of samples")
plt.title("Number of samples per species")
plt.show()
```

Output

```
<Figure size 600x400 with 1 Axes>
```

Code cell

```
# Task D3: average petal area by species
avg_petal_area = df.groupby("species")["petal_area"].mean()

plt.figure(figsize=(6, 4))
plt.bar(avg_petal_area.index, avg_petal_area.values)
plt.xlabel("Species")
plt.ylabel("Average petal area")
plt.title("Virginica has the largest average petal area")
plt.show()
```

Output

```
<Figure size 600x400 with 1 Axes>
```

Question D1: What does each point in a scatter plot represent?

Question D2: What does a histogram show that a mean value cannot show?

Task D4: Choose one plot and improve its title so that it explains the main pattern.

6. Part E - Visualization with Seaborn

Seaborn works well with Pandas and is useful for statistical plots.

Code cell

```
sns.scatterplot(data=df, x="petal_length", y="petal_width", hue="species")
plt.title("Petal length vs petal width by species")
plt.show()
```

Output

```
<Figure size 640x480 with 1 Axes>
```

Code cell

```
# Task E1: sepal_length vs sepal_width with species colors
sns.scatterplot(data=df, x="sepal_length", y="sepal_width", hue="species")
plt.title("Sepal length vs sepal width by species")
plt.show()
```

Output

```
<Figure size 640x480 with 1 Axes>
```

Code cell

```
sns.boxplot(data=df, x="species", y="petal_length")
plt.title("Petal length distribution by species")
plt.show()
```

Output

```
<Figure size 640x480 with 1 Axes>
```

Code cell

```
# Task E2: boxplot for sepal_width grouped by species
sns.boxplot(data=df, x="species", y="sepal_width")
plt.title("Sepal width distribution by species")
plt.show()
```

Output

```
<Figure size 640x480 with 1 Axes>
```

Code cell

```
numeric_df = df.select_dtypes(include="number")
corr = numeric_df.corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap="Blues", fmt=".2f")
plt.title("Correlation between numerical features")
plt.show()
```

Output

```
<Figure size 800x600 with 2 Axes>
```

Question E1: What does a boxplot help us compare between species?

Question E2: Does a strong correlation always mean that one variable causes the other? Explain briefly.

Optional: Run a pair plot if your computer is fast enough.

Code cell

```
# Optional pair plot. Uncomment the next two lines if you want to run it.
# sns.pairplot(df[["sepal_length", "sepal_width", "petal_length", "petal_width", "species"]], hue="species")
# plt.show()
```

7. Part F - Basic OOP for data analysis

The class below stores the dataset and provides reusable methods for common analysis tasks.

Code cell

```
class IrisAnalyzer:
    def __init__(self, data):
        self.data = data

    def show_shape(self):
        return self.data.shape

    def species_counts(self):
        return self.data["species"].value_counts()

    def feature_mean_by_species(self, feature):
        return self.data.groupby("species")[feature].mean()

    def feature_max_by_species(self, feature):
        return self.data.groupby("species")[feature].max()

    def filter_by_species(self, species_name):
        return self.data[self.data["species"] == species_name]

    def plot_feature_by_species(self, feature):
        sns.boxplot(data=self.data, x="species", y=feature)
        plt.title(f"{feature} by species")
        plt.show()

analyzer = IrisAnalyzer(df)
print(analyzer.show_shape())
print(analyzer.species_counts())
print(analyzer.feature_mean_by_species("petal_length"))
print(analyzer.feature_max_by_species("petal_length"))
analyzer.plot_feature_by_species("sepal_length")
```

Output

```
(150, 9)
species
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
species
setosa        1.462
versicolor    4.260
virginica     5.552
Name: petal_length, dtype: float64
species
setosa        1.9
versicolor    5.1
virginica     6.9
Name: petal_length, dtype: float64
```

Output

```
<Figure size 640x480 with 1 Axes>
```

Task F1: Add a method called `feature_max_by_species(self, feature)`.

Task F2: Add a method called `filter_by_species(self, species_name)`.

Task F3: Use the class to plot `sepal_length` by species.

Code cell

```
class IrisAnalyzerV2:
    def __init__(self, data):
        self.data = data.copy()

    def check_feature(self, feature):
        if feature not in self.data.columns:
            raise ValueError(f"Feature {feature} does not exist in the DataFrame.")

    def basic_summary(self):
        return self.data.describe()

    def mean_by_species(self, feature):
        self.check_feature(feature)
        return self.data.groupby("species")[feature].mean()

    def max_by_species(self, feature):
        self.check_feature(feature)
        return self.data.groupby("species")[feature].max()

    def min_by_species(self, feature):
        self.check_feature(feature)
        return self.data.groupby("species")[feature].min()

    def plot_boxplot(self, feature):
        self.check_feature(feature)
        sns.boxplot(data=self.data, x="species", y=feature)
        plt.title(f"{feature} by species")
        plt.show()

    def plot_histogram(self, feature):
        self.check_feature(feature)
        plt.figure(figsize=(6, 4))
        plt.hist(self.data[feature], bins=12)
        plt.xlabel(feature)
        plt.ylabel("Frequency")
        plt.title(f"Distribution of {feature}")
        plt.show()

analyzer2 = IrisAnalyzerV2(df)
print(analyzer2.mean_by_species("petal_area"))
print(analyzer2.min_by_species("petal_area"))
analyzer2.plot_histogram("petal_area")
```

Output

```
species
setosa         0.3656
versicolor     5.7204
virginica     11.2962
Name: petal_area, dtype: float64
species
setosa        0.11
versicolor    3.30
virginica     7.50
Name: petal_area, dtype: float64
```

Output

```
<Figure size 600x400 with 1 Axes>
```

Question F1: What is the role of `self` inside a Python class?

Question F2: Why is it better to put repeated analysis operations inside methods instead of copying the same code many times?

8. Part G - Connection to Lecture 1

Answer the following questions in markdown cells.

Question G1: Which part of this lab is closest to the data layer discussed in Lecture 1?

Question G2: Why should an AI student learn data inspection and visualization before learning model training?

Code cell

```
```